BRAIN TUMOR DETECTION AND LLM ASSISTANT

An AI-Based MRI Classification and Retrieval-Augmented Question Answering System

Project Title	Brain Tumor Detection and LLM Assistant Using Vision Transformer and RAG
Submitted By	Shafaat Ullah
Department	Artificial Intelligence
Development Platform	Google Colab / Python
Primary Framework	PyTorch, Hugging Face Transformers
Project Type	Educational / Research Prototype

PROJECT DOCUMENTATION

This document describes the project architecture, dataset, model workflow, chatbot retrieval pipeline, evaluation outputs, and limitations.

1. Project Overview

Brain Tumor Detection and LLM Assistant is an educational AI prototype that combines medical-image classification with a question-answering assistant. The image-classification component uses a pretrained Vision Transformer (ViT) to classify brain MRI images into four dataset categories. A separate language-model component uses retrieval-augmented generation (RAG) to answer general educational questions using a small knowledge base.

The two components serve different purposes: the ViT predicts an image class, while the LLM assistant retrieves relevant text and generates a text response. The chatbot does not interpret MRI images and the classifier does not provide a medical diagnosis.

2. Project Objectives

- Build an image-classification pipeline for four brain MRI dataset classes.
- Apply transfer learning with a pretrained Vision Transformer.
- Evaluate the classifier using accuracy, a classification report, and a confusion matrix.
- Create an educational question-answering assistant using FLAN-T5 Base.
- Implement semantic retrieval using Sentence Transformers and FAISS.
- Export test-set predictions and confidence scores to a downloadable CSV file.

3. Dataset Description

The project uses the Brain Tumor MRI Dataset hosted on Kaggle. The dataset is downloaded programmatically in the notebook using KaggleHub, rather than being uploaded manually. It contains separate Training and Testing directories, each organized into class folders.

Dataset source: Kaggle – Brain Tumor MRI Dataset

Class	Description
Glioma	MRI images labeled as glioma in the source dataset.
Meningioma	MRI images labeled as meningioma in the source dataset.
Pituitary Tumor	MRI images labeled as pituitary tumor in the source dataset.
No Tumor	MRI images labeled as no tumor in the source dataset.

The notebook obtains the actual class names from the dataset folder structure. Training data is split into training and validation subsets (85% / 15%), while the separate Testing folder is used for final evaluation.

4. Image Classification Methodology

4.1 Preprocessing

- Resize MRI images to 224 × 224 pixels.
- Convert images to tensors.
- Normalize RGB channels using ImageNet mean and standard deviation.
- Apply random horizontal flipping and small random rotations to training images only.
- Use deterministic evaluation transforms for validation and testing.

4.2 Model Architecture

The classifier uses the Vision Transformer Base (ViT-B/16) architecture with pretrained ImageNet weights from torchvision. The original classification head is replaced with a new linear layer containing four output units, one for each dataset class. In the initial training stage, the pretrained backbone is frozen and the new classification head is trained.

4.3 Training

- Framework: PyTorch.
- Loss function: CrossEntropyLoss.
- Optimizer: AdamW.
- Initial learning rate: 0.001 for the classification head.
- Weight decay: 0.0001.
- Batch size: 16.
- The notebook trains for five epochs and saves the checkpoint with the best validation accuracy.

4.4 Evaluation

The saved best-validation checkpoint is loaded for evaluation on the held-out Testing directory. The notebook calculates test accuracy, precision, recall and F1-score, and plots a confusion matrix. A separate demonstration cell selects a random test MRI image and displays its actual class, predicted class and model confidence.

5. LLM Assistant and RAG Methodology

The assistant is a separate text-based component. It uses Google FLAN-T5 Base to generate answers and a retrieval pipeline to supply relevant educational context.

5.1 Knowledge Base

The current notebook uses a small built-in educational knowledge base containing introductory notes about brain tumors, glioma, meningioma, pituitary tumors, MRI, possible symptoms, treatment categories, and limitations of AI classification. These notes are starter content and are not a comprehensive medical reference library.

5.2 Text Embeddings and Vector Search

- Sentence Transformer: all-MiniLM-L6-v2.
- Text is divided into manageable chunks with overlap.
- Each chunk is converted into a normalized semantic embedding.
- FAISS IndexFlatIP stores the vectors and retrieves the top three most similar chunks for a user question.

5.3 Answer Generation

The retrieved text is added to a prompt instructing FLAN-T5 to answer only from the supplied context, acknowledge missing information, and avoid diagnosing patients or prescribing treatment. The generated answer is returned together with the retrieved source labels.

RAG workflow: User question → embedding generation → FAISS similarity search → relevant text chunks → FLAN-T5 prompt → generated educational response.

6. Overall System Workflow

The notebook is organized as a sequential Google Colab workflow:

- Install required Python packages and import libraries.
- Download the Kaggle MRI dataset programmatically.
- Inspect the Training and Testing folder structure and visualize sample images.
- Preprocess images and create training, validation and testing data loaders.
- Load the pretrained ViT model, replace its classification head, and train.
- Plot training history and evaluate the best model on the test set.
- Run a sample-image prediction and display class confidence.
- Build the educational knowledge base, text embeddings and FAISS index.
- Load FLAN-T5 Base and test the RAG question-answering function.
- Save model and retrieval artifacts, generate prediction CSV, and download it.

7. Generated Files and Outputs

File / Output	Purpose
best_brain_vit.pth / brain_tumor_vit.pth	Trained ViT model checkpoint / saved model state.
brain_tumor_faiss.index	FAISS vector index used for semantic retrieval.
rag_documents.json	Knowledge-base document content and source labels.
evaluation.txt	Saved evaluation summary and classification metrics.
brain_tumor_predictions.csv	Test-image names, actual labels, predicted labels, confidence and correctness.

8. Software and Libraries

Component	Technology
Notebook environment	Google Colab
Programming language	Python
Image classification	PyTorch, torchvision, Vision Transformer
Language model	Hugging Face Transformers, google/flan-t5-base
Embeddings	Sentence Transformers, all-MiniLM-L6-v2
Vector database / search	FAISS (IndexFlatIP)
Evaluation / visualization	scikit-learn, Matplotlib, Seaborn
Dataset download	KaggleHub

9. Results and Performance Reporting

The notebook calculates performance metrics when executed. Since the final numerical results depend on the actual training run, GPU availability, package versions and dataset files, this documentation does not invent or assume an accuracy value.

Record the values printed by the final summary cell in the following table after running all notebook cells successfully:

Metric	Run result
Best validation accuracy	Enter value from notebook output
Test accuracy	Enter value from notebook output
Precision / Recall / F1-score	Use per-class and macro/weighted values from classification report
Dataset image counts	Use training, validation and testing counts printed by final summary

10. Limitations

- The model is trained on one public dataset; performance may not generalize to other hospitals, scanners, populations or MRI protocols.
- The initial training configuration freezes the ViT backbone and trains only the classification head for a short run. Fine-tuning, more epochs and systematic hyperparameter testing may improve or change results.
- The chatbot knowledge base is small and manually written for demonstration. It does not provide verified, comprehensive or patient-specific medical advice.
- The language model may generate incomplete or inaccurate responses even when supplied with retrieved context.
- The classifier predicts one of the dataset labels; it does not determine tumor grade, clinical severity, treatment or prognosis.

11. Future Improvements

- Fine-tune selected ViT layers and compare performance with other pretrained architectures.
- Use stratified validation, class-wise analysis, data augmentation experiments and cross-validation where appropriate.
- Add explainability methods such as attention visualization or Grad-CAM-style analysis suitable for the chosen architecture.
- Expand the RAG knowledge base using reliable, reviewed medical and academic sources with source citations and update dates.
- Add a user interface that clearly separates image classification from text-based educational assistance.
- Test robustness, calibration, data leakage, class imbalance and external generalization before making any real-world claims.

12. Conclusion

This project demonstrates how computer vision and language technologies can be combined in one educational prototype. A Vision Transformer performs four-class brain MRI image classification, while a retrieval-augmented FLAN-T5 assistant answers general questions from a small text knowledge base. The project provides practical experience with transfer learning, model evaluation, semantic embeddings, vector search and language-model generation.

13. Important Medical Disclaimer

This project is for educational and research purposes only. It is not a medical device and must not be used to diagnose, rule out, or treat brain tumors. Predictions can be wrong. Any health concern or MRI interpretation must be discussed with a qualified medical professional, such as a radiologist or neurologist.

Prepared by: Shafaat Ullah

Project: Brain Tumor Detection and LLM Assistant Using Vision Transformer and RAG